CMPE 273 · FINAL PROJECT · SPRING 2026

EraseGraph

Verifiable deletion propagation
across distributed systems.

Frontend
React Dashboard

Backend
Orchestrator (NestJS)

RabbitMQ
Event Bus + DLQ

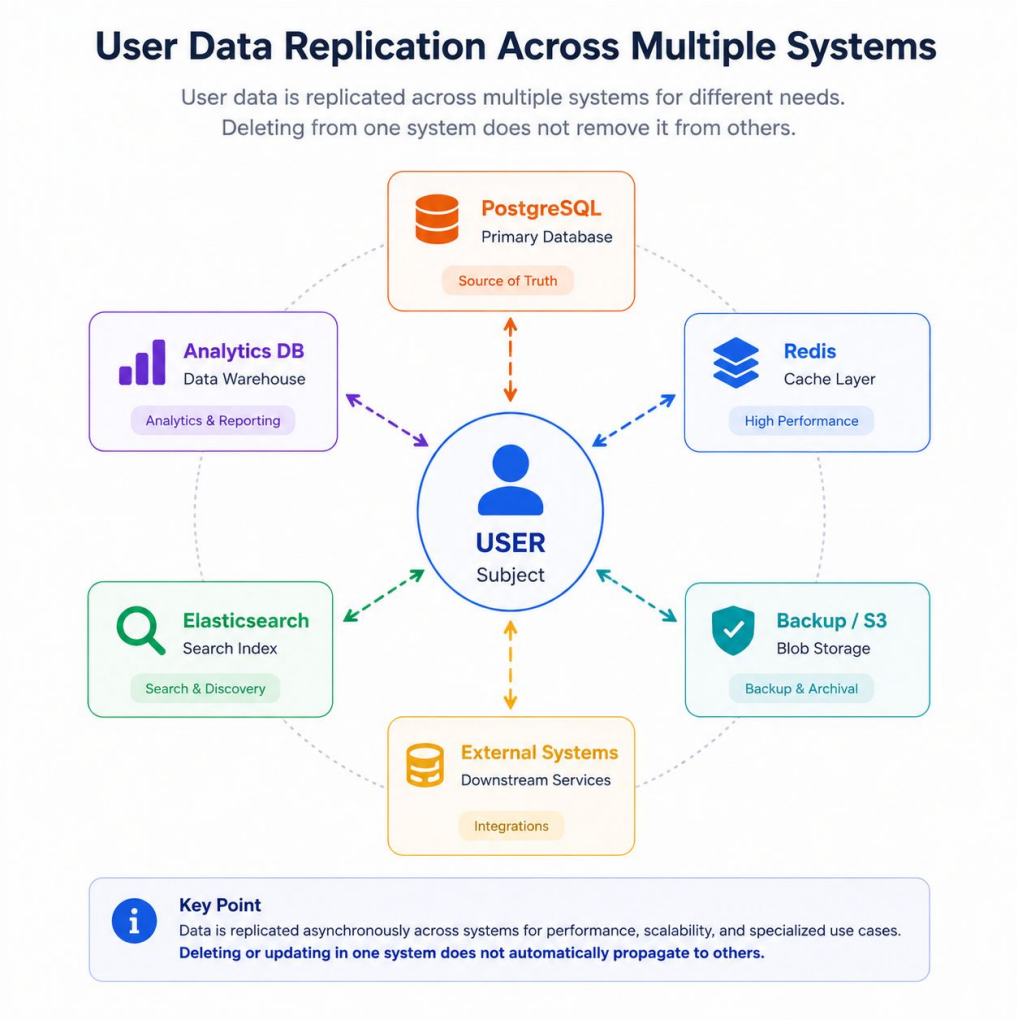
Workers
5 Cleanup + Proof

TEAM · GROUP 6

Haoyuan Shan · Vritika Malhotra · Sakshat Patil · Asim Mohammed

The Deletion Problem

User data lives in many systems at once. Removing it from one leaves stale copies everywhere else.



Data is scattered everywhere

Every user record exists in the database, the cache, search, analytics, and backups simultaneously.

Deleting from one place is not enough

Removing a row from Postgres still leaves copies in Redis, Elasticsearch, S3, and analytics.

Compliance asks for proof

GDPR Article 17 and CCPA need real evidence that deletion actually propagated across the stack.

Partial failures stay hidden

Without tracing, when one service fails silently the rest looks fine — and stale data lingers.

Target Users & Use Cases

WHO

Privacy / Compliance Teams

WHY

Regulatory compliance needs

WHERE

SaaS, FinTech, Healthcare

WHO

Platform Engineering

WHY

Data consistency assurance

WHERE

Enterprise Microservices

WHO

SRE / DevOps Teams

WHY

Operational observability

WHERE

Cloud platforms

WHO

Admin Operators

WHY

Manual deletion management

WHERE

Internal tools

CONCRETE EXAMPLE

Mike

Data Protection Engineer

ROLE & PROBLEM

Ensures deletion compliance at fintech company. Manual deletion takes 2-3 weeks with high audit burden.

SOLUTION

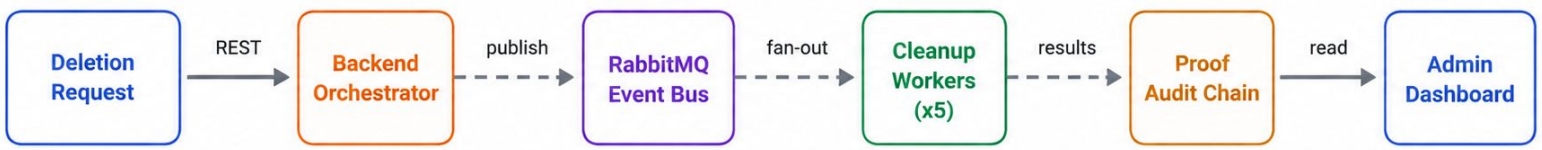
EraseGraph reduces time to **30 seconds** + cryptographic proof.

IMPACT

Compliance confirmed, audit-ready, **$50K risk eliminated**.

Our Solution: Event-Driven Orchestration

One API request fans out to every data store in parallel — with a cryptographic audit trail.



Async event-driven workflow with verifiable deletion proof.

One request, five systems

- Single REST call kicks off the workflow
- 5 cleanup workers run in parallel via RabbitMQ
- Backend aggregates everything into one status

Built to handle failures

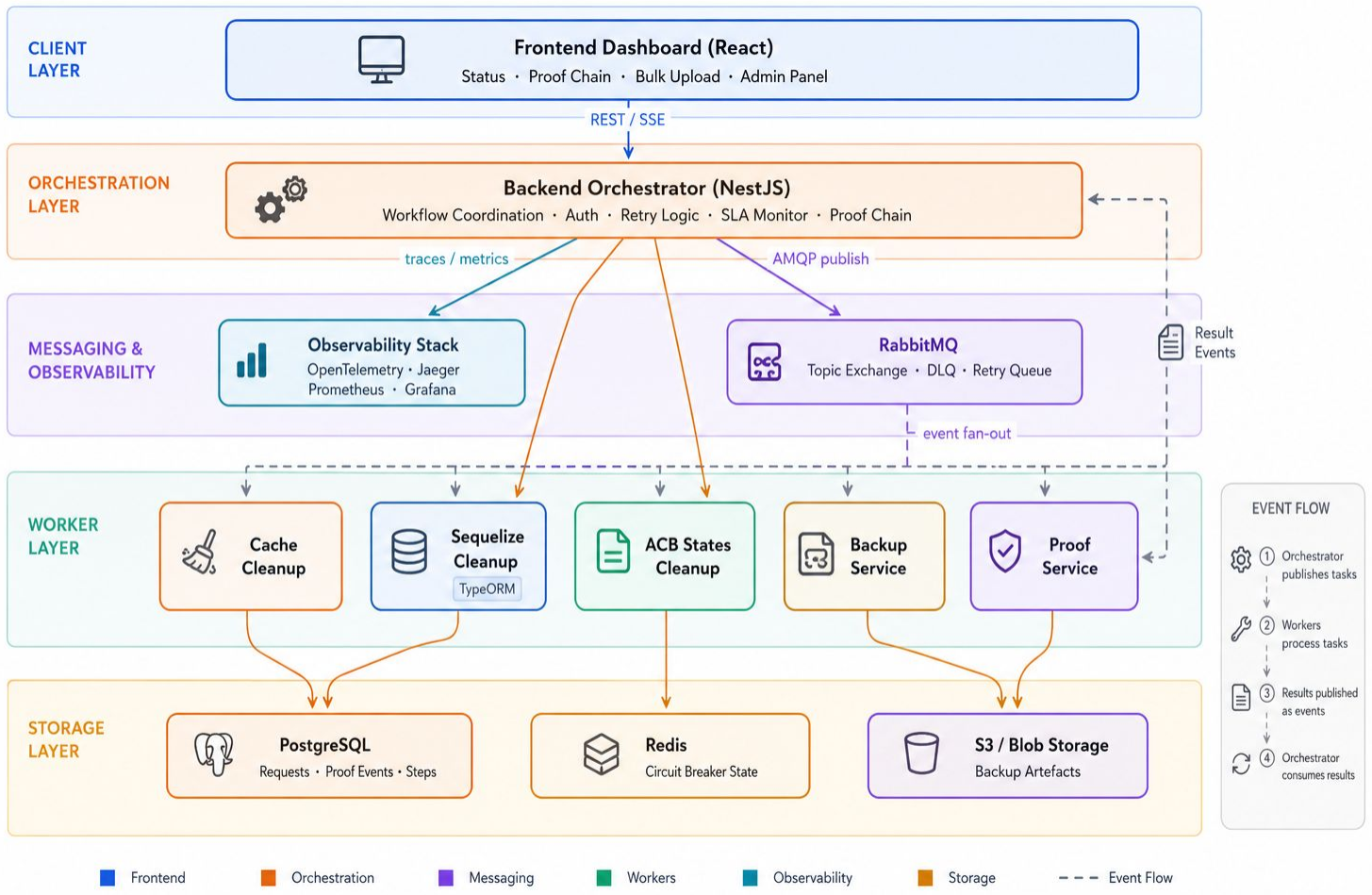
- Auto-retry with TTL queues (5s / 10s / 20s)
- DLQ catches anything that fails 3 times
- Circuit breakers stop cascade failures

Proof you can verify

- Every event SHA-256 hash-chained
- Tamper any record and verify fails
- Export the full chain as JSON

System Architecture

Five cooperating layers — frontend, orchestration, messaging, workers, and storage.



LAYERED BREAKDOWN

CLIENT

React dashboard — status, proof, bulk upload, admin panel.

ORCHESTRATION

NestJS backend — workflow, auth, retry, SLA monitor, proof chain.

MESSAGING

RabbitMQ topic exchange, DLQ, retry queues, event fan-out.

WORKERS

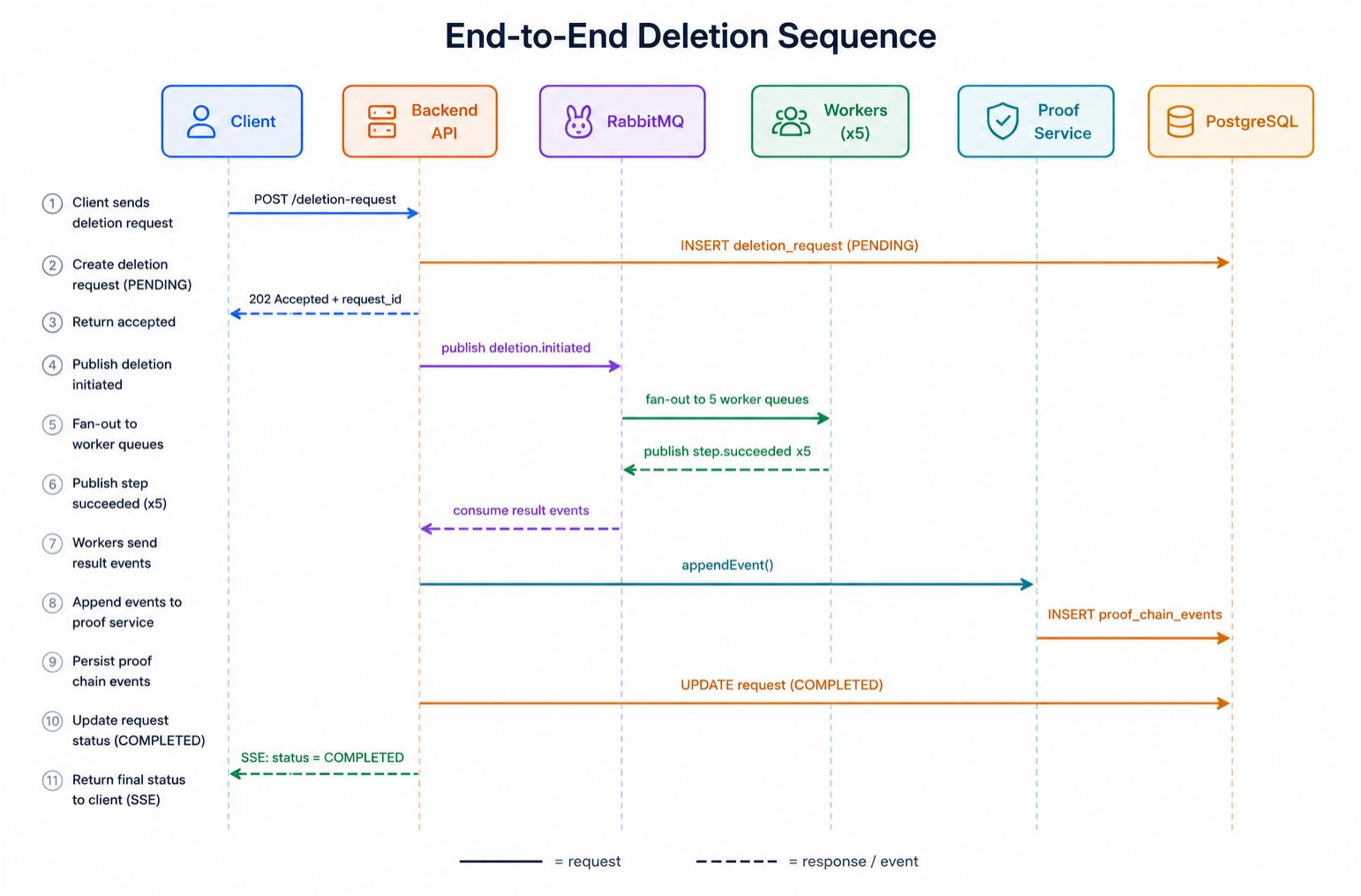
5 cleanup services (cache, search, primary, analytics, backup) + proof service.

STORAGE

Postgres (state + proof events) · Redis (CB state) · S3 (backup artefacts).

Request Sequence — Step by Step

Eleven steps grouped into three phases — from REST call to SSE completion event.



THREE PHASES

1 Request Intake

Steps 1–3

Client POSTs deletion request. Backend inserts a PENDING row and returns 202 with request_id.

2 Fan-Out + Workers

Steps 4–7

Backend publishes deletion.initiated. RabbitMQ fans out to 5 workers — each reports step.succeeded back.

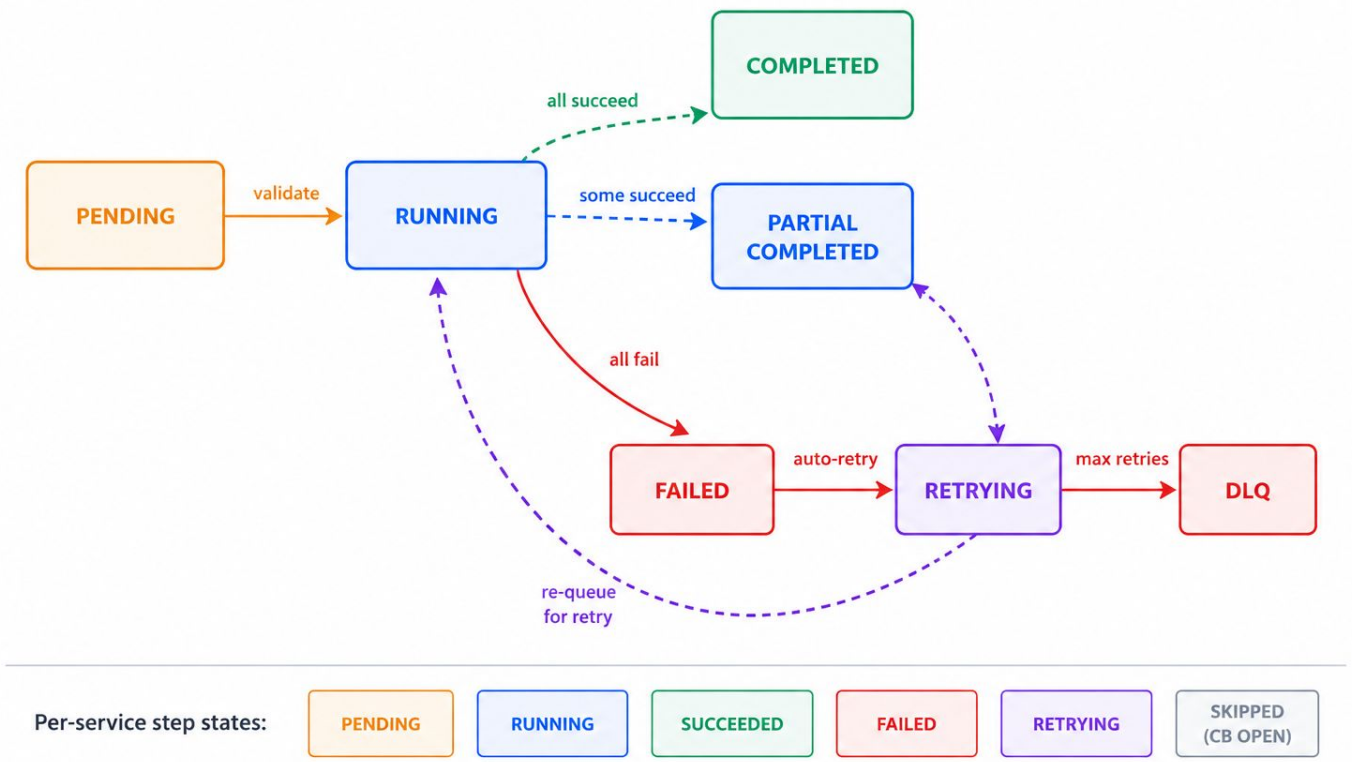
3 Proof + Completion

Steps 8–11

Proof service appends hash-chained events. Backend marks request COMPLETED and streams final SSE.

Request Lifecycle & State Machine

Every deletion moves through a clear state machine with step-level visibility.



How states flow

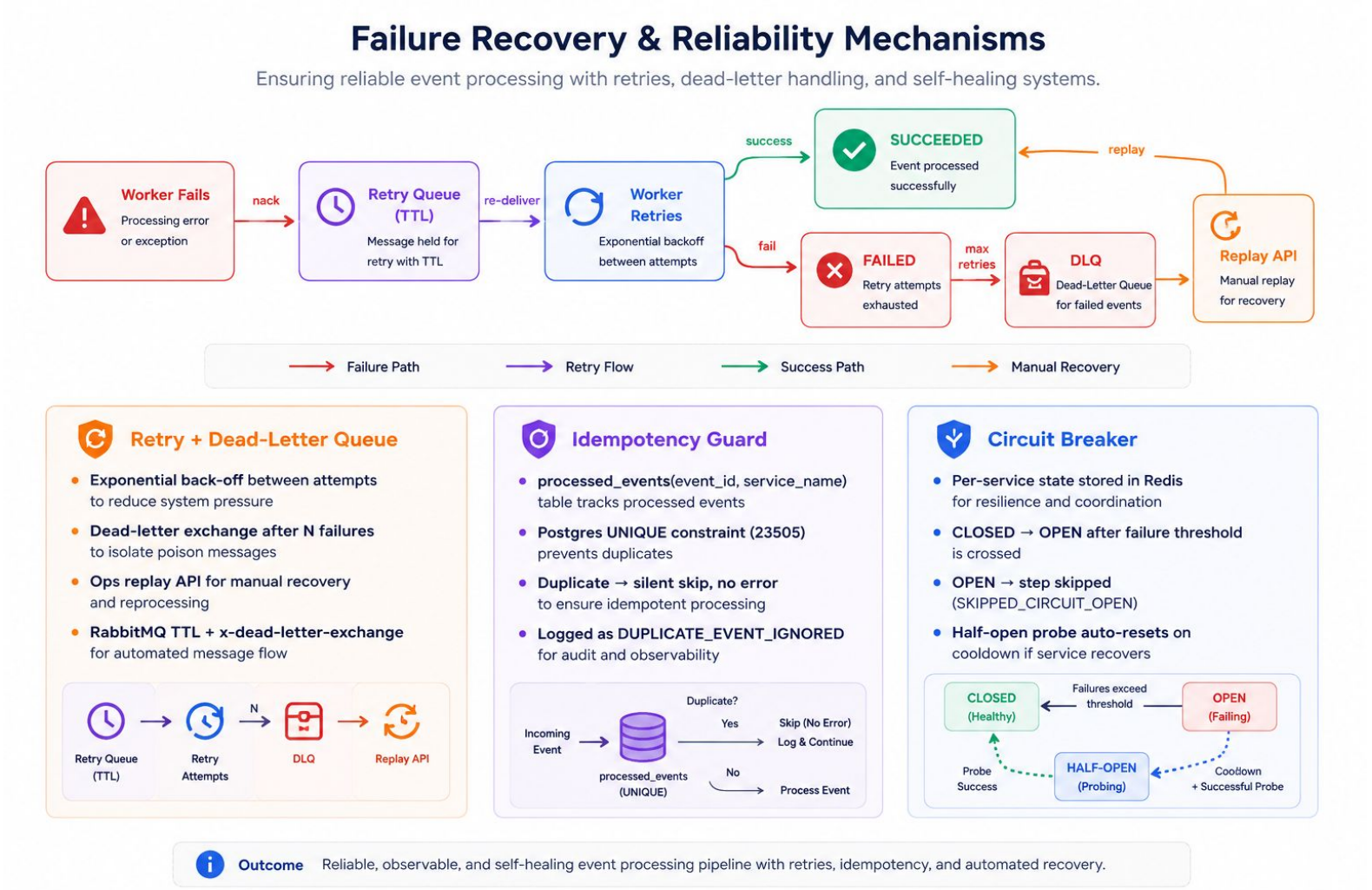
Every worker reports its own status: PENDING → RUNNING → SUCCEEDED or FAILED. The backend rolls these step-level updates into one overall request status visible end-to-end.

Why PARTIAL_COMPLETED exists

Slow services like analytics or backup do not block primary deletion. Auditors see exactly which systems were cleaned — far more useful than a binary pass / fail.

Reliability Design

Three layered mechanisms make sure every deletion eventually completes — or is explicitly accounted for.



How Reliability Works — Three Layers

Retry, idempotency, and circuit breaker chain together to absorb failures without losing work.

1 Retry + DLQ

RabbitMQ TTL queues

When a worker fails, the message is requeued with backoff.

- Retries at 5s → 10s → 20s TTLs
- After 3 fails → moved to DLQ
- Ops can manually replay DLQ

2 Idempotency Guard

Postgres unique constraint

Duplicate events are detected and ignored safely.

- processed_events table tracks event_id
- Duplicate insert → silent skip, no error
- Logs DUPLICATE_EVENT_IGNORED for audit

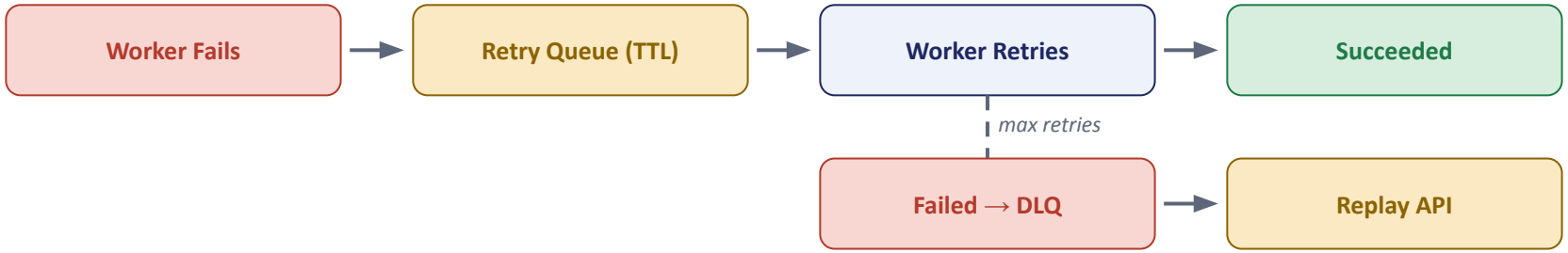
3 Circuit Breaker

Redis state per service

Repeated failures trip the breaker and isolate the failure.

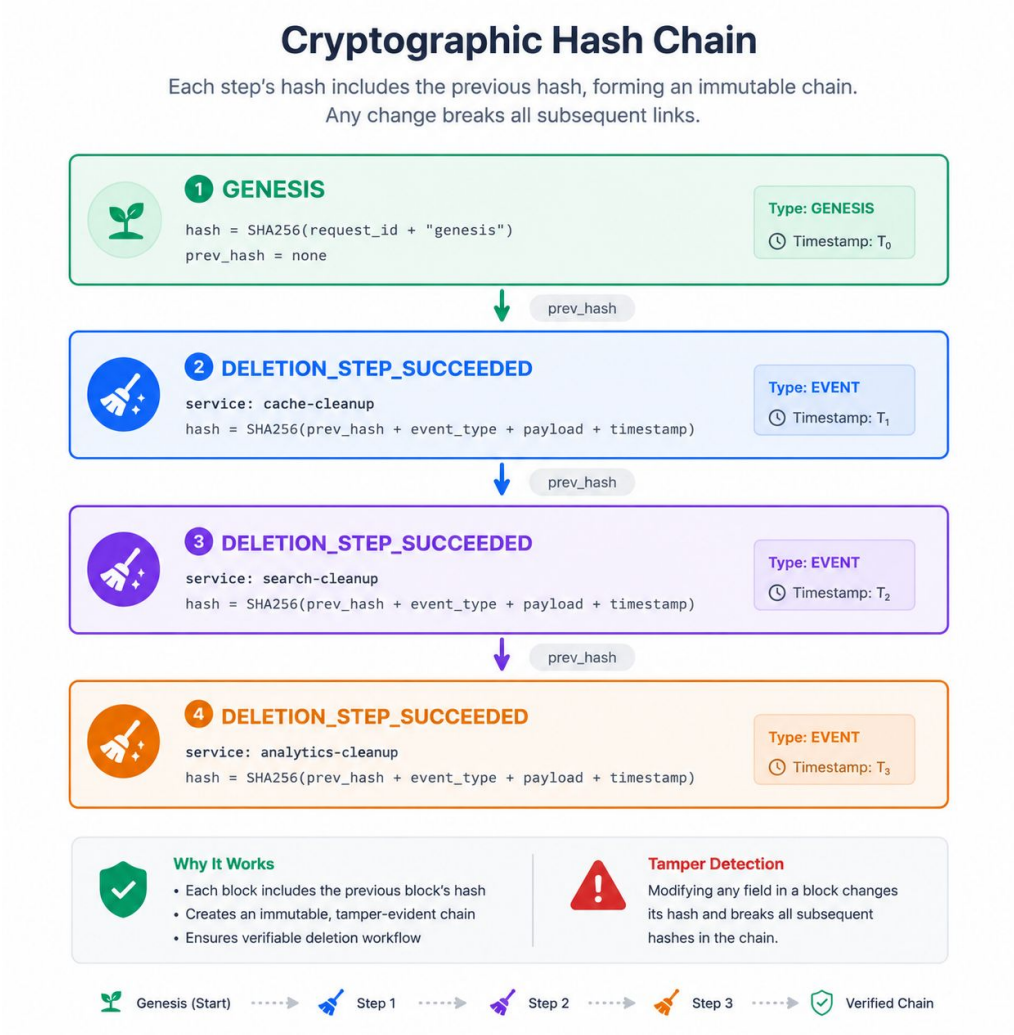
- CLOSED → OPEN after 3 consecutive fails
- OPEN window = 30s, then HALF-OPEN probe
- Half-open probe success → back to CLOSED

FAILURE RECOVERY FLOW



Proof & Auditability

Every deletion event is hash-chained. Modify any record and verification fails — cryptographically guaranteed.



Each event chains to the last

Every proof event stores SHA-256(prev_hash + type + payload + timestamp). Change one record and the chain breaks from there on.

Safe under concurrent writes

Postgres advisory locks serialise hash-chain writes — no race conditions even when multiple workers report at once.

One-click verification

Export the full chain as JSON. Re-run verify and every record reports VERIFIED or TAMPERED.

Built for compliance

Audit-ready evidence for GDPR Article 17 and CCPA deletion requirements — straight out of the box.

Consistency & Tradeoffs

Not every data store needs the same consistency guarantee — each one picks the right tradeoff.

Data Store	Deletion Strategy	Latency Impact	Consistency Model	Staleness Risk
PostgreSQL	Synchronous DELETE	Low	Strong (ACID)	LOW — Best Consistency
Redis Cache	Async AMQP Event	Very Low	Eventual	VERY LOW — Cache TTL Safety
Search Index	Async AMQP Event	Low	Eventual	LOW — Reindex on Lag
Analytics DB	Soft-delete / Delayed	High	Eventual	HIGHER — Intentional Delay
Backup / S3	Scheduled / Async	High	Eventual	HIGHER — Scheduled Run

Why PARTIAL_COMPLETED is good design

If analytics or backup is slow, primary deletion still completes. Auditors see exactly which systems were cleaned — far more useful than a binary pass / fail.

Where we sit on CAP

We favour AP (Availability + Partition Tolerance) for the event-driven path. The hash-chained proof chain gives us consistency through eventual convergence and tamper detection.

Observability & Operations

Every request, event, and failure is traced, measured, and surfaced in real time.

Distributed Tracing

OpenTelemetry | Jaeger

- Traces follow a request across every service
- W3C trace-context headers carry through RabbitMQ
- Spot exactly where slowness or failures happen

Metrics & Dashboards

Prometheus | Grafana

- Throughput, step duration, retry counts, CB state
- Per-service histograms and error rates
- Live dashboards for ops, refreshed on the fly

Real-Time SSE Streaming

Server-Sent Events

- Frontend gets live status — no polling
- Each request opens its own SSE channel
- Backend closes the stream on COMPLETED / FAILED

Admin Dashboard

Live Operator View

- Per-service health: healthy / degraded / down
- Circuit breaker state — CLOSED / OPEN / HALF-OPEN
- SLA violations + one-click DLQ replay

Kubernetes-Ready, Cloud-Deployed

Every service is containerised — local via Docker Compose, production on Kubernetes / GKE.

Local — Docker Compose

One command brings up the whole stack: backend, frontend, 5 cleanup workers, proof, Postgres, Redis, RabbitMQ, plus Jaeger and Prometheus.

Production — Kubernetes

Deployment manifests, Services, and ConfigMaps for every component. Workers scale horizontally based on queue depth.

Cloud — Live on GKE

Running on a real GKE cluster, all services verified end-to-end. Secrets via .env.example — zero credentials committed to git.

DEPLOYMENT AT A GLANCE

10+	Microservices
5	Data Stores
3	Reliability Layers
100%	End-to-end Verified

Key Results

What we shipped and demonstrated across the full semester — by the numbers.

10+	5	3	15	65
Microservices	Data Stores	Reliability Layers	Test Suites	Unit Tests

Complete Distributed System

- 10+ services coordinating deletion across 5 data stores
- Event-driven workers, independently deployable
- Full lifecycle — request intake to audit export

Verifiable Proof Chain

- SHA-256 hash chain for every deletion request
- Tampering detected down to a single record
- One-click verify returns VERIFIED or TAMPERED

Failure-Safe by Design

- Retry + DLQ + idempotency + circuit breaker
- No deletion lost silently — even under partitions
- DLQ replay lets ops recover any failed step

EraseGraph shows that compliance-grade deletion across distributed systems is achievable — with event-driven orchestration, layered reliability, and cryptographic auditability.

Challenges & Lessons Learned

Building reliable distributed systems is harder than it looks — here is what we learned along the way.

Distributed Idempotency

- Same event arriving twice could cause double deletes
- Solved with a Postgres unique constraint on event_id
- Duplicate events are silently skipped and logged

Hash Chain Concurrency

- Two workers writing at once could corrupt prev_hash
- pg_advisory_xact_lock serialises chain writes
- Verified correct under load with integration tests

Circuit Breaker Persistence

- Breaker state has to survive worker restarts
- Stored in Redis with TTL — survives across pods
- Half-open probes required careful timeout tuning

Trace Context Across Async

- OpenTelemetry spans were lost across RabbitMQ
- We propagate W3C trace-context manually in headers
- Jaeger now shows full end-to-end traces per request

KEY LESSON *Event-driven fan-out beats synchronous RPC for deletion — looser coupling, better scalability, and clearer failure visibility.*

Future Work

Three directions to take EraseGraph from a proof-of-concept to a production-grade platform.

1 Scale It Up

Move from RabbitMQ to Apache Kafka

- Persistent, replayable event log
- Consumer-group auto-scaling with KEDA on K8s
- Partition deletes by user_id for parallelism
- Exactly-once delivery via Kafka transactions

2 Automate Compliance

GDPR / CCPA out of the box

- Auto-generated deletion certificate (PDF)
- Email SLA-violation reports to data officers
- Multi-region residency tagging per request
- Webhook notifications when deletion completes

3 Smarter Resilience

Observability-driven optimisation

- ML anomaly detection on deletion latency
- GraphQL audit-query API for flexible reporting
- Chaos-engineering suite (fault injection)
- Auto-replay when SLA breach is detected

Team Contributions

Who owned what and the impact each member delivered.

	OWNED	KEY DELIVERABLES	IMPACT
Haoyuan Shan *Integration & Deployment*	Kubernetes + cloud deployment, backend integration, repo consolidation	K8s manifests, GKE deployment, proof-chain optimisation, demo data	*Made the project demo-ready and submission-ready end to end*
Vritika Malhotra *Reliability Engineering*	Retry queues, DLQ, idempotency, circuit breaker, bulk CSV upload	Retry + DLQ flow, idempotency guard, CB service, /deletions/bulk	*Made deletion robust to failures and added bulk operator workflows*
Sakshat Patil *Frontend & Real-Time UX*	Dashboard, SSE streaming, proof timeline, history + admin pages	React pages, SSE EventSource, proof export, admin live view	*Turned the backend pipeline into a clear, real-time operator view*
Asim Mohammed *Consistency & Admin Workflow*	Consistency semantics, state model, SLA monitor, admin endpoints	SLA scanner + violations API, status transitions, state design	*Made the state model clearer and operationally reliable*

Thank you.

Questions?

Verifiable deletion propagation · CMPE 273 · Spring 2026

Haoyuan Shan · Vritika Malhotra · Sakshat Patil · Asim Mohammed